

# Using Activation Functions in Deep Learning Models

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch**  2

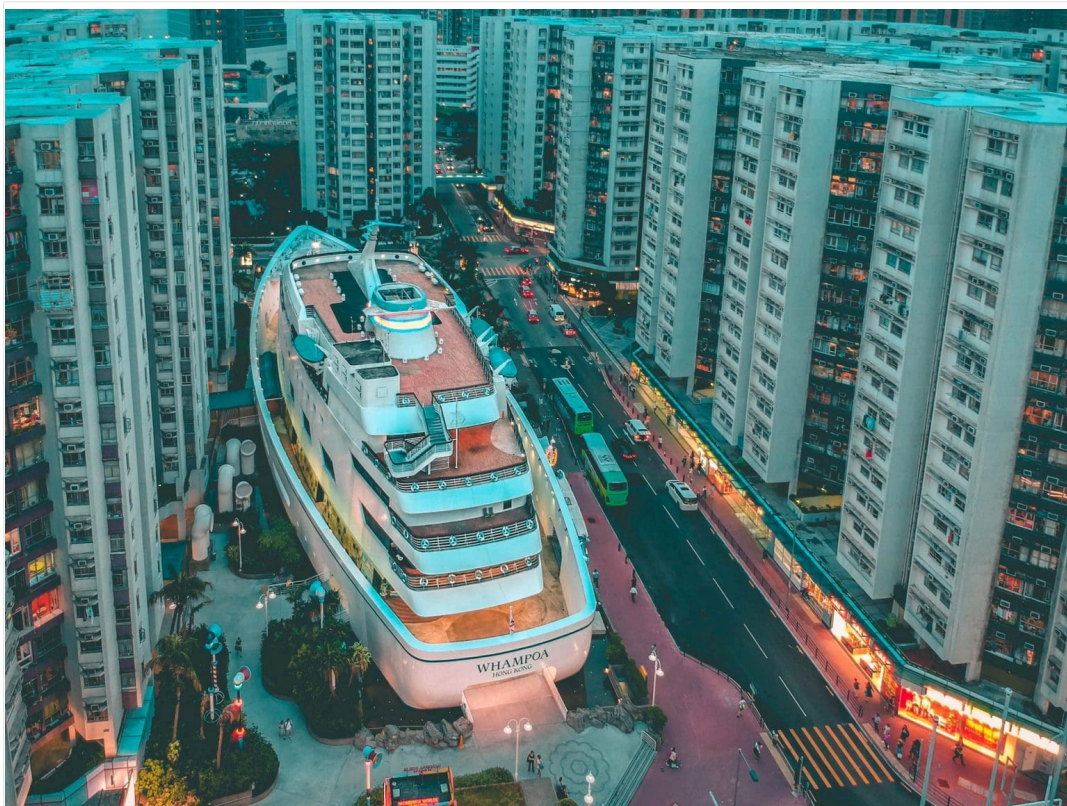
 Share  Share  Post

A deep learning model in its simplest form are layers of perceptrons connected in tandem. Without any activation functions, they are just matrix multiplications with limited power, regardless how many of them. Activation is the magic why neural network can be an approximation to a wide variety of non-linear function. In PyTorch, there are many activation functions available for use in your deep learning models. In this post, you will see how the choice of activation functions can impact the model. Specifically,

- What are the common activation functions
- What are the nature of activation functions
- How the different activation functions impact the learning rate
- How the selection of activation function can solve the vanishing gradient problem

**Kick-start your project** with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Using Activation Functions in Deep Learning Models  
Photo by SHUJA OFFICIAL. Some rights reserved.

## Overview

This post is in three parts; they are

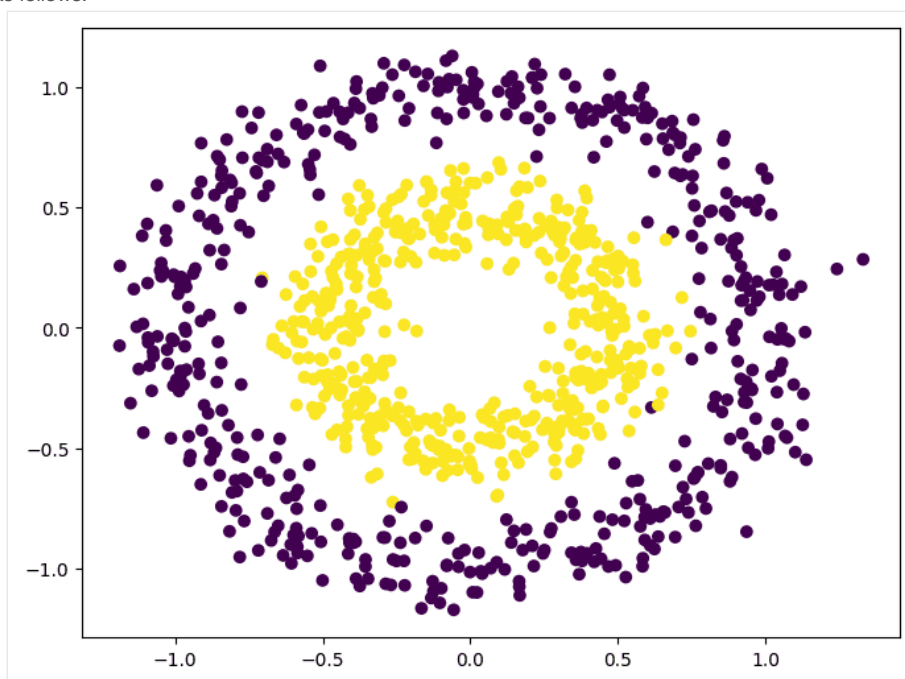
- A Toy Model of Binary Classification
- Why Nonlinear Functions?
- The Effect of Activation Functions

## A Toy Model of Binary Classification

Let's start with a simple example of binary classification. Here you use the `make_circles()` function from scikit-learn to create a synthetic dataset for binary classification. This dataset has two features: The x- and y-coordinate of points. Each point belongs to one of the two classes. You can generate 1000 data points and visualize them as below:

```
1 from sklearn.datasets import make_circles
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6
7 # Make data: Two circles on x-y plane as a classification problem
8 X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
9 X = torch.tensor(X, dtype=torch.float32)
10 y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)
11
12 plt.figure(figsize=(8,6))
13 plt.scatter(X[:,0], X[:,1], c=y)
14 plt.show()
```

The dataset is visualized as follows:



This dataset is special because it is simple but not linearly separable: It is impossible to find a straight line to separate two classes. How can your neural network figure out there's a circle boundary between the classes is a challenge.

Let's create a deep learning model for this problem. To make things simple, you do not do cross validation. You may find the neural network overfit the data but it doesn't affect the discussion below. The model has 4 hidden layers and the output layer gives a sigmoidal value (0 to 1) for binary classification. The model accepts a parameter at its constructor to specify what is the activation to use in the hidden layers. You implement the training loop in a function as you will run this for several times.

The implementation is as follows:

```
1 class Model(nn.Module):
2     def __init__(self, activation=nn.ReLU):
3         super().__init__()
4         self.layer0 = nn.Linear(2,5)
5         self.act0 = activation()
6         self.layer1 = nn.Linear(5,5)
7         self.act1 = activation()
8         self.layer2 = nn.Linear(5,5)
9         self.act2 = activation()
10        self.layer3 = nn.Linear(5,5)
11        self.act3 = activation()
12        self.layer4 = nn.Linear(5,1)
13        self.act4 = nn.Sigmoid()
14
15    def forward(self, x):
```

```

16     x = self.act0(self.layer0(x))
17     x = self.act1(self.layer1(x))
18     x = self.act2(self.layer2(x))
19     x = self.act3(self.layer3(x))
20     x = self.act4(self.layer4(x))
21     return x
22
23 def train_loop(model, X, y, n_epochs=300, batch_size=32):
24     loss_fn = nn.BCELoss()
25     optimizer = optim.Adam(model.parameters(), lr=0.0001)
26     batch_start = torch.arange(0, len(X), batch_size)
27
28     bce_hist = []
29     acc_hist = []
30
31     for epoch in range(n_epochs):
32         # train model with optimizer
33         model.train()
34         for start in batch_start:
35             X_batch = X[start:start+batch_size]
36             y_batch = y[start:start+batch_size]
37             y_pred = model(X_batch)
38             loss = loss_fn(y_pred, y_batch)
39             optimizer.zero_grad()
40             loss.backward()
41             optimizer.step()
42         # evaluate BCE and accuracy at end of each epoch
43         model.eval()
44         with torch.no_grad():
45             y_pred = model(X)
46             bce = float(loss_fn(y_pred, y))
47             acc = float((y_pred.round() == y).float().mean())
48         bce_hist.append(bce)
49         acc_hist.append(acc)
50         # print metrics every 10 epochs
51         if (epoch+1) % 10 == 0:
52             print("Before epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch+1, bce, acc*100))
53     return bce_hist, acc_hist

```

At the end of each epoch in the training function, you evaluate the model with the entire dataset. The evaluation result is returned when the training finished. In the following, you create a model, train it, and plot the training history. The activation function you use is **rectified linear unit** or ReLU, which is the most common activation function nowadays:

```

1 activation = nn.ReLU
2 model = Model(activation=activation)
3 bce_hist, acc_hist = train_loop(model, X, y)
4 plt.plot(bce_hist, label="BCE")
5 plt.plot(acc_hist, label="Accuracy")
6 plt.xlabel("Epochs")
7 plt.ylim(0, 1)
8 plt.title(str(activation))
9 plt.legend()
10 plt.show()

```

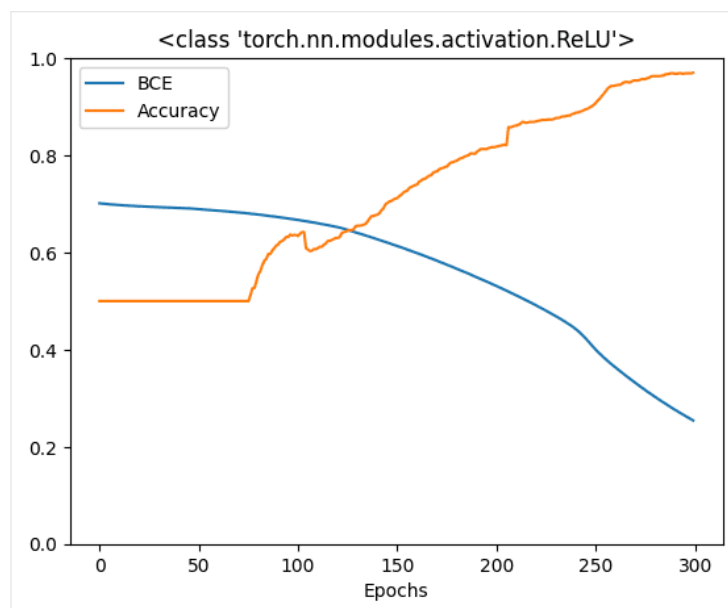
Running this give you the following:

```

1 Before epoch 10: BCE=0.7025, Accuracy=50.00%
2 Before epoch 20: BCE=0.6990, Accuracy=50.00%
3 Before epoch 30: BCE=0.6959, Accuracy=50.00%
4 ...
5 Before epoch 280: BCE=0.3051, Accuracy=96.30%
6 Before epoch 290: BCE=0.2785, Accuracy=96.90%
7 Before epoch 300: BCE=0.2543, Accuracy=97.00%

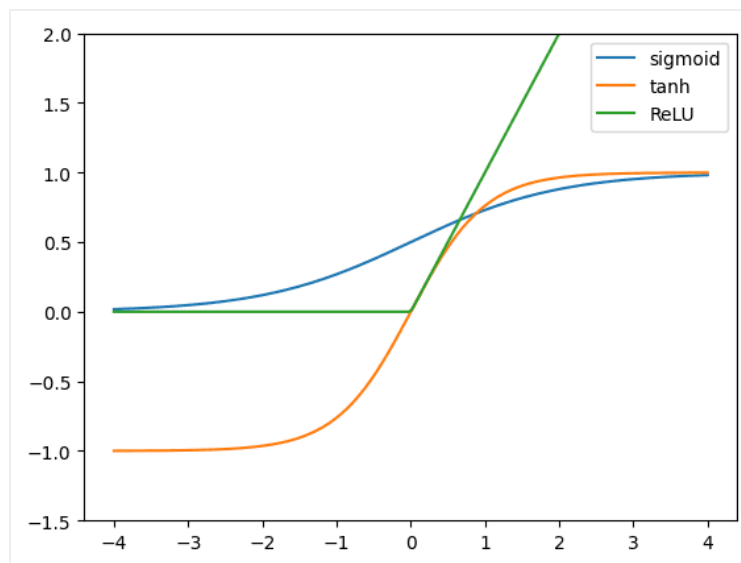
```

and this plot:



This model works great. After 300 epochs, it can achieve 90% accuracy. However, ReLU is not the only activation function. Historically, sigmoid function and hyperbolic tangents were common in neural networks literatures. If you're curious, below are how you can compare these three activation functions, using matplotlib:

```
1 x = torch.linspace(-4, 4, 200)
2 relu = nn.ReLU()(x)
3 tanh = nn.Tanh()(x)
4 sigmoid = nn.Sigmoid()(x)
5
6 plt.plot(x, sigmoid, label="sigmoid")
7 plt.plot(x, tanh, label="tanh")
8 plt.plot(x, relu, label="ReLU")
9 plt.ylim(-1.5, 2)
10 plt.legend()
11 plt.show()
```



ReLU is called rectified linear unit because it is a linear function  $y = x$  at positive  $x$  but remains zero if  $x$  is negative. Mathematically, it is  $y = \max(0, x)$ .

Hyperbolic tangent ( $y = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ) goes from -1 to +1 smoothly while sigmoid function ( $y = \sigma(x) = \frac{1}{1 + e^{-x}}$ ) goes from 0 to +1.

If you try to differentiate these functions, you will find that ReLU is the easiest: The gradient is 1 at positive region and 0 otherwise. Hyperbolic tangent has a steeper slope therefore its gradient is greater than that of sigmoid function.

All these functions are increasing. Therefore, their gradients are never negative. This is one of the criteria for an activation function suitable to use in neural networks.

## Want to Get Started With Deep Learning with PyTorch?

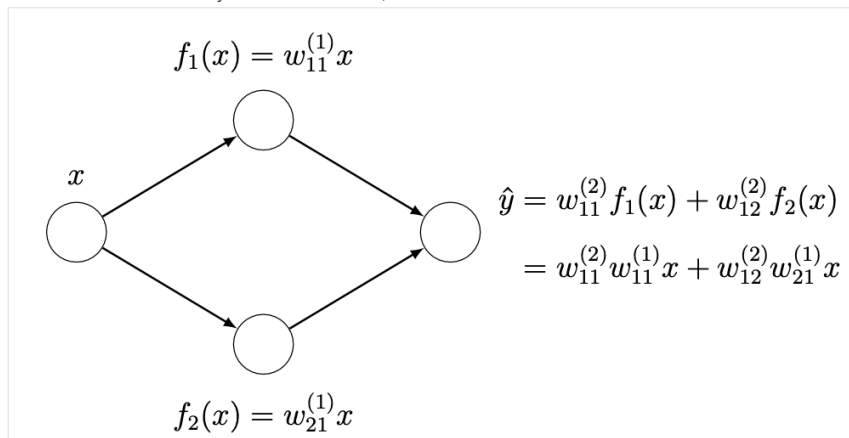
Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

## Why Nonlinear Functions?

You might be wondering, why all this hype about nonlinear activation functions? Or why can't we just use an identity function after the weighted linear combination of activations from the previous layer? Using multiple linear layers is basically the same as using a single linear layer. This can be seen through a simple example. Let's say you have a one hidden layer neural network, each with two hidden neurons.



You can then rewrite the output layer as a linear combination of the original input variable if you used a linear hidden layer. If you had more neurons and weights, the equation would be a lot longer with more nesting and more multiplications between successive layer weights. However, the idea remains the same: You can represent the entire network as a single linear layer. To make the network represent more complex functions, you would need nonlinear activation functions.

## The Effect of Activation Functions

To explain how much impact the activation function can bring to your model, let's modify the training loop function to capture more data: The gradients in each training step. Your model has four hidden layers and one output layer. In each step, the backward pass calculates the gradient of the weights of each layer and the weight update is done by the optimizer based on the result of the backward pass. You should observe how the gradient changes as the training progressed. Therefore, the training loop function is modified to collect the mean absolute value of the gradient in each layer in each step, as follows:

```
1 def train_loop(model, X, y, n_epochs=300, batch_size=32):
2     loss_fn = nn.BCELoss()
3     optimizer = optim.Adam(model.parameters(), lr=0.0001)
4     batch_start = torch.arange(0, len(X), batch_size)
5
6     bce_hist = []
7     acc_hist = []
8     grad_hist = [[], [], [], [], []]
9
10    for epoch in range(n_epochs):
11        # train model with optimizer
12        model.train()
13        layer_grad = [[], [], [], [], []]
14        for start in batch_start:
15            X_batch = X[start:start+batch_size]
16            y_batch = y[start:start+batch_size]
17            y_pred = model(X_batch)
18            loss = loss_fn(y_pred, y_batch)
19            optimizer.zero_grad()
20            loss.backward()
21            optimizer.step()
22            # collect mean absolute value of gradients
23            layers = [model.layer0, model.layer1, model.layer2, model.layer3, model.layer4]
24            for n, layer in enumerate(layers):
25                mean_grad = float(layer.weight.grad.abs().mean())
26                layer_grad[n].append(mean_grad)
27        # evaluate BCE and accuracy at end of each epoch
28        model.eval()
29        with torch.no_grad():
30            y_pred = model(X)
```

```

31     bce = float(loss_fn(y_pred, y))
32     acc = float((y_pred.round() == y).float().mean())
33     bce_hist.append(bce)
34     acc_hist.append(acc)
35     for n, grads in enumerate(layer_grad):
36         grad_hist[n].append(sum(grads)/len(grads))
37     # print metrics every 10 epochs
38     if epoch % 10 == 9:
39         print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
40     return bce_hist, acc_hist, layer_grad

```

At the end of the inner for-loop, the gradients of the layer weights are computed by the backward process earlier and you can access to the gradient using `model.layer0.weight.grad`. Like the weights, the gradients are tensors. You take the absolute value of each element and then compute the mean over all elements. This value depends on the batch and can be very noisy. Thus you summarize all such mean absolute value over the same epoch at the end.

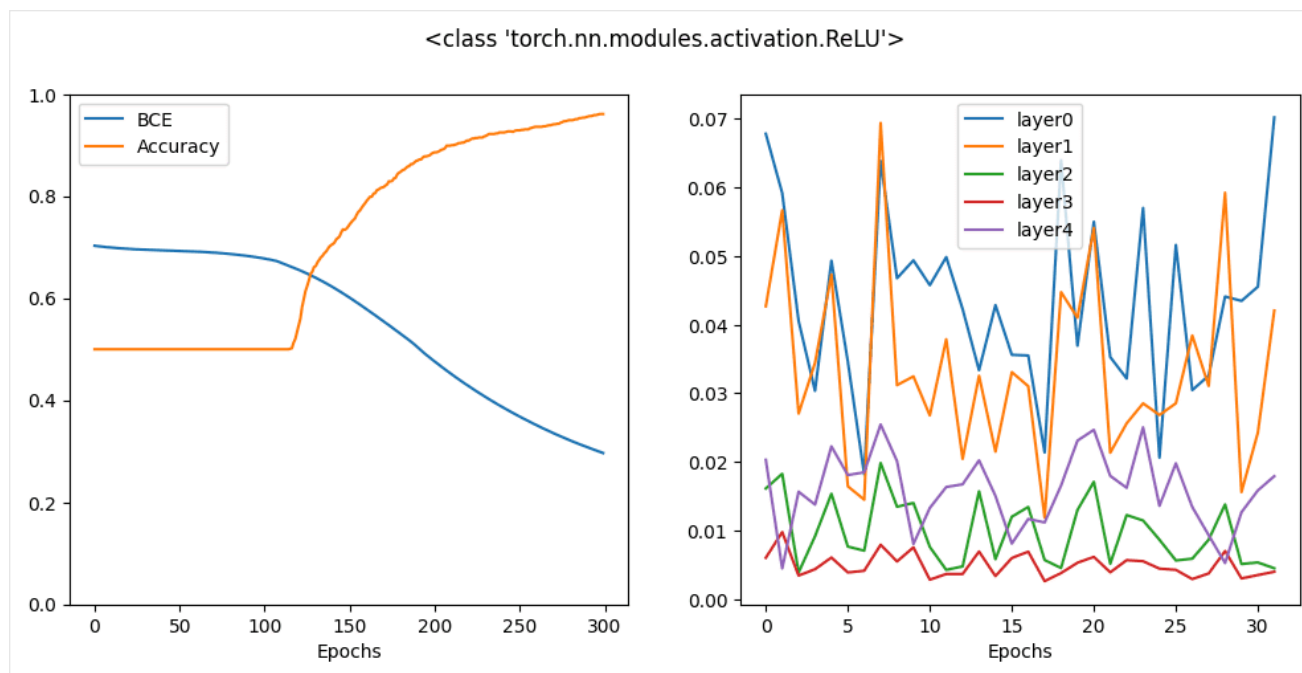
Note that you have five layers in the neural network (hidden and output layers combined). So you can see the pattern of each layer's gradient across the epochs if you visualize them. In below, you run the training loop as before and plot both the cross entropy and accuracy as well as the mean absolute gradient of each layer:

```

1 activation = nn.ReLU
2 model = Model(activation=activation)
3 bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
4
5 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
6 ax[0].plot(bce_hist, label="BCE")
7 ax[0].plot(acc_hist, label="Accuracy")
8 ax[0].set_xlabel("Epochs")
9 ax[0].set_ylim(0, 1)
10 for n, grads in enumerate(grad_hist):
11     ax[1].plot(grads, label="layer"+str(n))
12 ax[1].set_xlabel("Epochs")
13 fig.suptitle(str(activation))
14 ax[0].legend()
15 ax[1].legend()
16 plt.show()

```

Running the above produces the following plot:



In the plot above, you can see how the accuracy increases and the cross entropy loss decreases. At the same time, you can see the gradient of each layer is fluctuating in a similar range, especially you should pay attention to the line corresponding to the first layer and the last layer. This behavior is ideal.

Let's repeat the same with a sigmoid activation:

```

1 activation = nn.Sigmoid
2 model = Model(activation=activation)
3 bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
4
5 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
6 ax[0].plot(bce_hist, label="BCE")
7 ax[0].plot(acc_hist, label="Accuracy")
8 ax[0].set_xlabel("Epochs")
9 ax[0].set_ylim(0, 1)
10 for n, grads in enumerate(grad_hist):
11     ax[1].plot(grads, label="layer"+str(n))
12 ax[1].set_xlabel("Epochs")

```

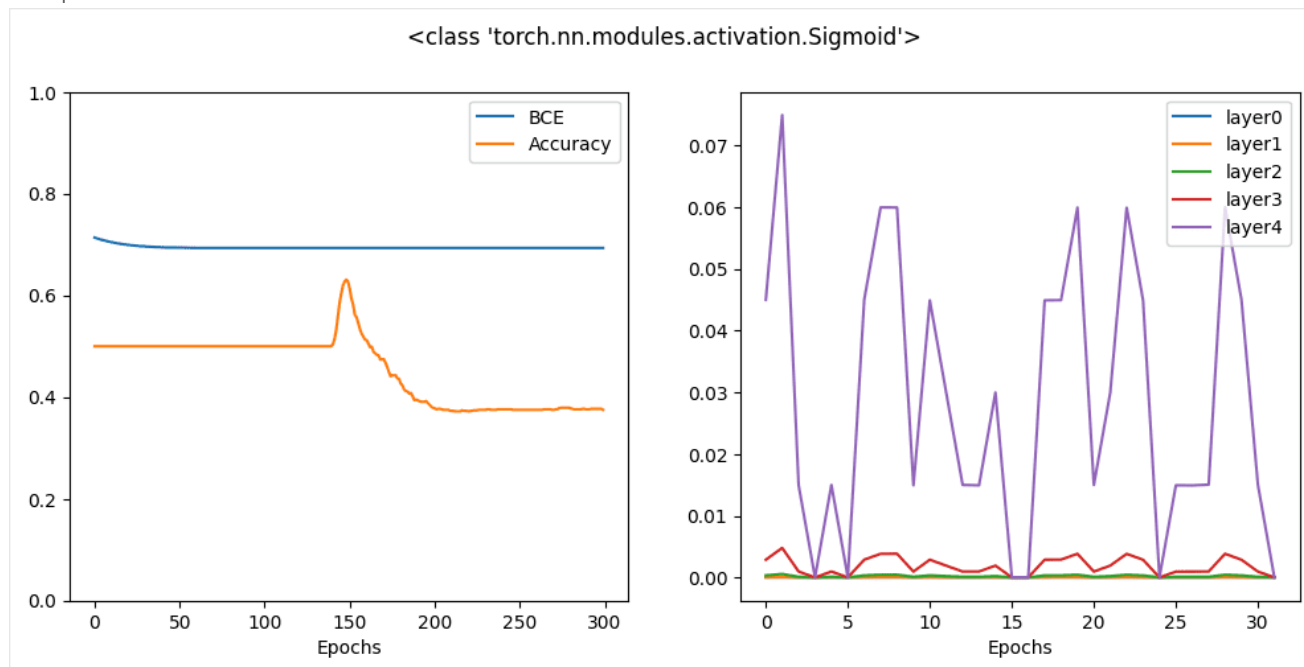


```

13 fig.suptitle(str(activation))
14 ax[0].legend()
15 ax[1].legend()
16 plt.show()

```

which the plot is as follows:



You can see that after 300 epochs, the final result is much worse than ReLU activation. Indeed, you may need much more epochs for this model to converge. The reason can be easily found on the graph at right, which you can see the gradient is significant only for the output layer while all the hidden layers' gradients are virtually zero. This is the **vanishing gradient effect** which is the problem of many neural network models with sigmoid activation function.

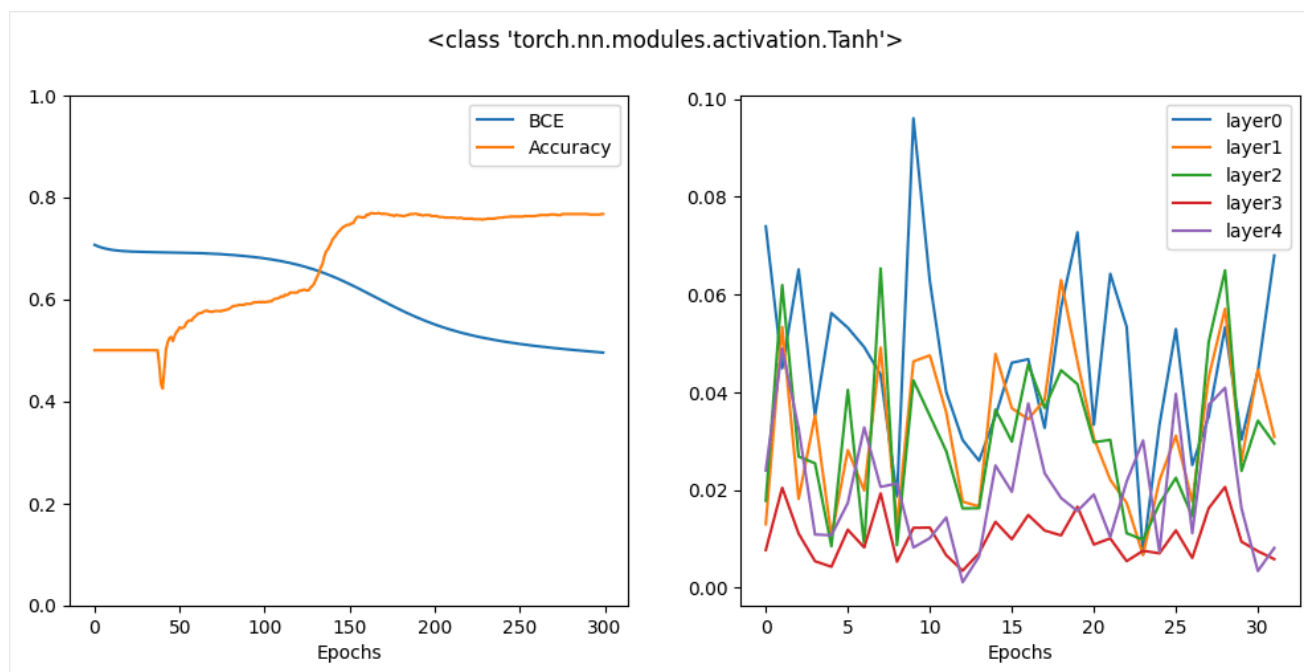
The hyperbolic tangent function has a similar shape as sigmoid function but its curve is steeper. Let's see how it behaves:

```

1 activation = nn.Tanh
2 model = Model(activation=activation)
3 bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
4
5 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
6 ax[0].plot(bce_hist, label="BCE")
7 ax[0].plot(acc_hist, label="Accuracy")
8 ax[0].set_xlabel("Epochs")
9 ax[0].set_ylim(0, 1)
10 for n, grads in enumerate(grad_hist):
11     ax[1].plot(grads, label="layer"+str(n))
12 ax[1].set_xlabel("Epochs")
13 fig.suptitle(str(activation))
14 ax[0].legend()
15 ax[1].legend()
16 plt.show()

```

Which is:

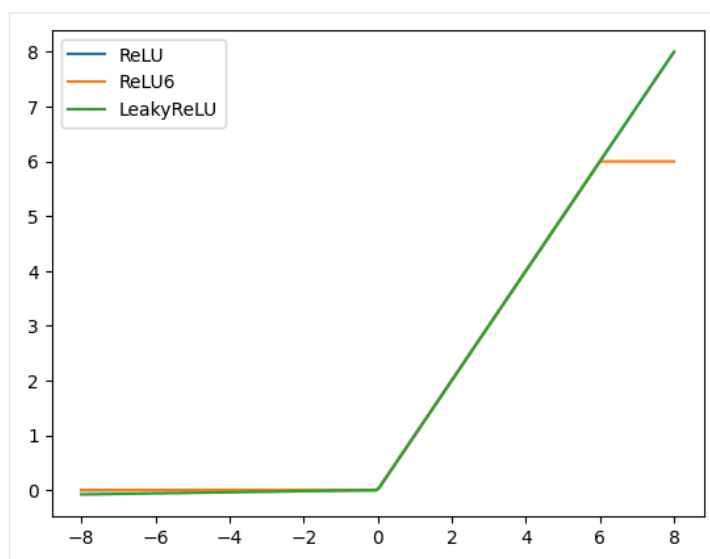


The result looks better than sigmoid activation but still worse than ReLU. In fact, from the gradient plot, you can notice that the gradients at the hidden layers are significant but the gradient at the first hidden layer is obviously at an order of magnitude less than that at the output layer. Thus the backward process is not very effective at propagating the gradient to the input.

This is the reason you see ReLU activation in every neural network model today. Not only because ReLU is simpler and computing the differentiation of it is much faster than the other activation function, but also because it can make the model converge faster.

Indeed, you can do better than ReLU sometimes. In PyTorch, you have a number of ReLU variations. Let's look at two of them. You can compare these three variations of ReLU as follows:

```
1 x = torch.linspace(-8, 8, 200)
2 relu = nn.ReLU()(x)
3 relu6 = nn.ReLU6()(x)
4 leaky = nn.LeakyReLU()(x)
5
6 plt.plot(x, relu, label="ReLU")
7 plt.plot(x, relu6, label="ReLU6")
8 plt.plot(x, leaky, label="LeakyReLU")
9 plt.legend()
10 plt.show()
```



First is the ReLU6, which is ReLU but cap the function at 6.0 if the input to the function is more than 6.0:

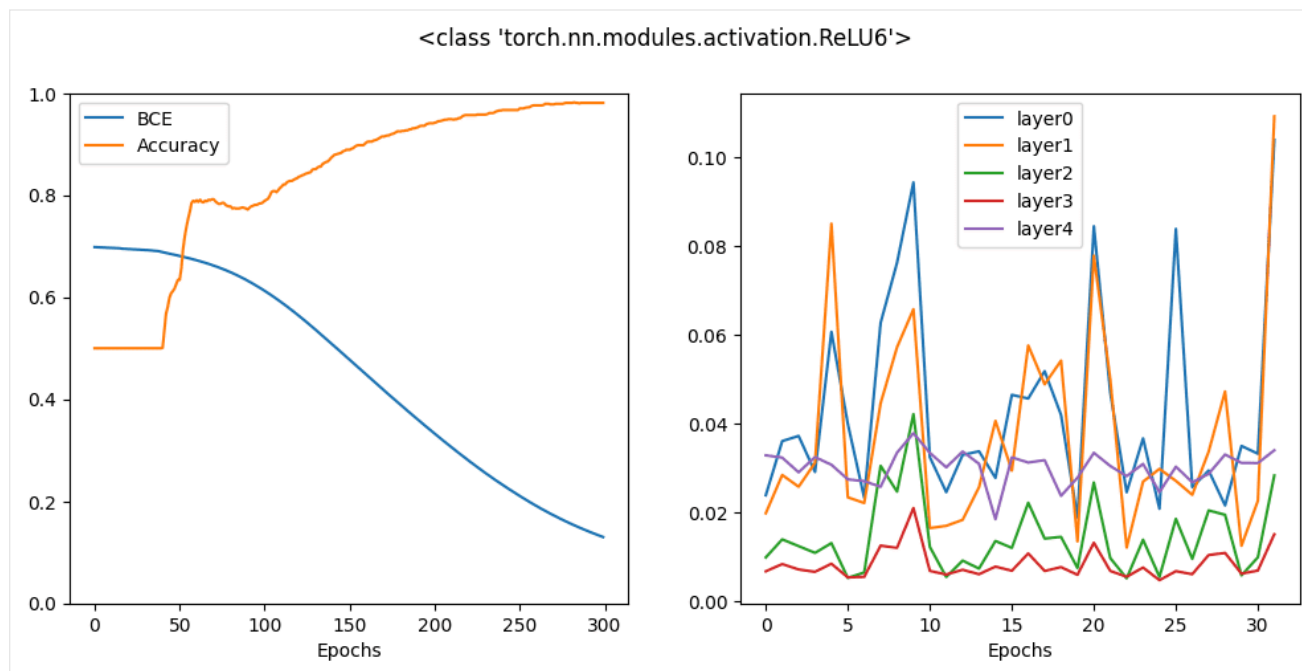
```
1 activation = nn.ReLU6
2 model = Model(activation=activation)
3 bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
4
```



```

5 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
6 ax[0].plot(bce_hist, label="BCE")
7 ax[0].plot(acc_hist, label="Accuracy")
8 ax[0].set_xlabel("Epochs")
9 ax[0].set_ylim(0, 1)
10 for n, grads in enumerate(grad_hist):
11     ax[1].plot(grads, label="layer"+str(n))
12 ax[1].set_xlabel("Epochs")
13 fig.suptitle(str(activation))
14 ax[0].legend()
15 ax[1].legend()
16 plt.show()

```

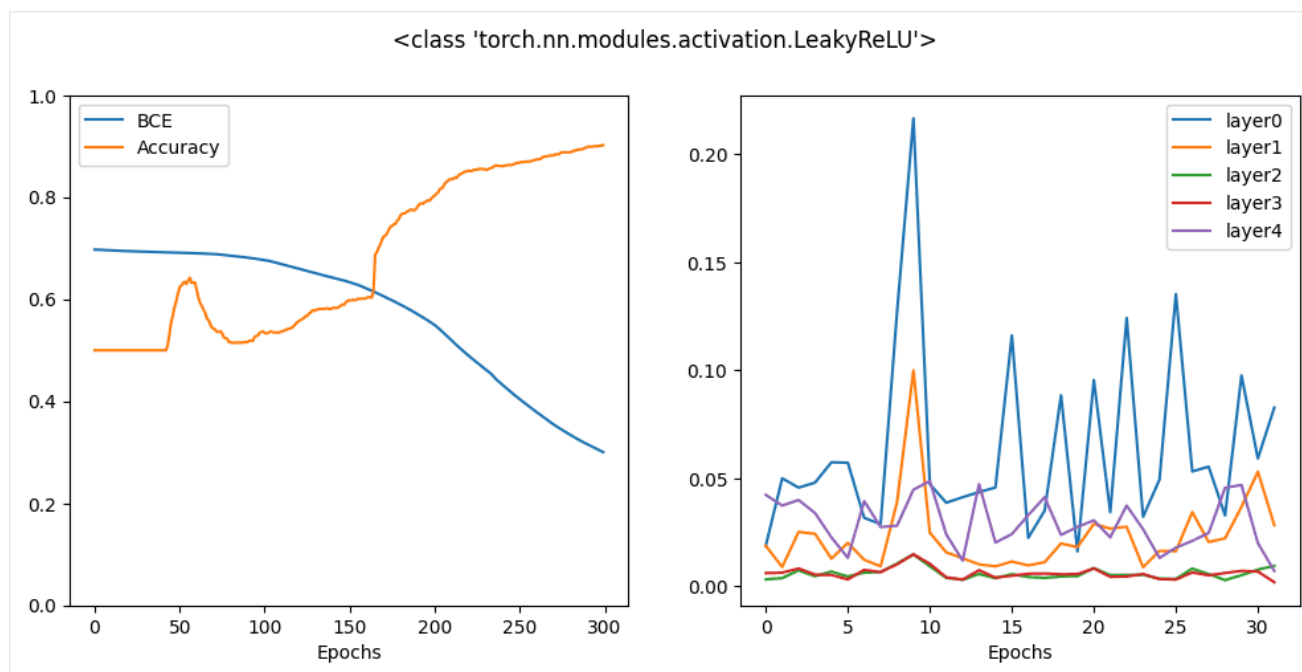


Next is leaky ReLU, which the negative half of ReLU is no longer flat but a gently slanted line. The rationale behind is to keep a small positive gradient at that region.

```

1 activation = nn.LeakyReLU
2 model = Model(activation=activation)
3 bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
4
5 fig, ax = plt.subplots(1, 2, figsize=(12, 5))
6 ax[0].plot(bce_hist, label="BCE")
7 ax[0].plot(acc_hist, label="Accuracy")
8 ax[0].set_xlabel("Epochs")
9 ax[0].set_ylim(0, 1)
10 for n, grads in enumerate(grad_hist):
11     ax[1].plot(grads, label="layer"+str(n))
12 ax[1].set_xlabel("Epochs")
13 fig.suptitle(str(activation))
14 ax[0].legend()
15 ax[1].legend()
16 plt.show()

```



You can see that all these variations can give you similar accuracy after 300 epochs but from the history curve, you know some are faster to reach a high accuracy than another. This is because of the interaction between the gradient of an activation function with the optimizer. There's no golden rule that a single activation function works best but the design helps:

- in backpropagation, passing the loss metric from the output layer all the way to the input layer
- maintaining stable gradient calculation under specific condition, e.g., limiting floating point precision
- providing enough contrast on different input such that the backward pass can determine accurate adjustment to the parameter

The following is the complete code to generate all the plots above:

```
1 from sklearn.datasets import make_circles
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6
7 # Make data: Two circles on x-y plane as a classification problem
8 X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
9 X = torch.tensor(X, dtype=torch.float32)
10 y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)
11
12 # Binary classification model
13 class Model(nn.Module):
14     def __init__(self, activation=nn.ReLU):
15         super().__init__()
16         self.layer0 = nn.Linear(2,5)
17         self.act0 = activation()
18         self.layer1 = nn.Linear(5,5)
19         self.act1 = activation()
20         self.layer2 = nn.Linear(5,5)
21         self.act2 = activation()
22         self.layer3 = nn.Linear(5,5)
23         self.act3 = activation()
24         self.layer4 = nn.Linear(5,1)
25         self.act4 = nn.Sigmoid()
26
27     def forward(self, x):
28         x = self.act0(self.layer0(x))
29         x = self.act1(self.layer1(x))
30         x = self.act2(self.layer2(x))
31         x = self.act3(self.layer3(x))
32         x = self.act4(self.layer4(x))
33         return x
34
35 # train the model and produce history
36 def train_loop(model, X, y, n_epochs=300, batch_size=32):
37     loss_fn = nn.BCELoss()
38     optimizer = optim.Adam(model.parameters(), lr=0.0001)
39     batch_start = torch.arange(0, len(X), batch_size)
40
41     bce_hist = []
42     acc_hist = []
43     grad_hist = [[],[],[],[],[]]
44
45     for epoch in range(n_epochs):
46         # train model with optimizer
47         model.train()
48         layer_grad = [[],[],[],[],[]]
```

```

49     for start in batch_start:
50         X_batch = X[start:start+batch_size]
51         y_batch = y[start:start+batch_size]
52         y_pred = model(X_batch)
53         loss = loss_fn(y_pred, y_batch)
54         optimizer.zero_grad()
55         loss.backward()
56         optimizer.step()
57         # collect mean absolute value of gradients
58         layers = [model.layer0, model.layer1, model.layer2, model.layer3, model.layer4]
59         for n, layer in enumerate(layers):
60             mean_grad = float(layer.weight.grad.abs().mean())
61             layer_grad[n].append(mean_grad)
62     # evaluate BCE and accuracy at end of each epoch
63     model.eval()
64     with torch.no_grad():
65         y_pred = model(X)
66         bce = float(loss_fn(y_pred, y))
67         acc = float((y_pred.round() == y).float().mean())
68     bce_hist.append(bce)
69     acc_hist.append(acc)
70     for n, grads in enumerate(layer_grad):
71         grad_hist[n].append(sum(grads)/len(grads))
72     # print metrics every 10 epochs
73     if epoch % 10 == 9:
74         print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
75     return bce_hist, acc_hist, layer_grad
76
77 # pick different activation functions and compare the result visually
78 for activation in [nn.Sigmoid, nn.Tanh, nn.ReLU, nn.ReLU6, nn.LeakyReLU]:
79     model = Model(activation=activation)
80     bce_hist, acc_hist, grad_hist = train_loop(model, X, y)
81
82     fig, ax = plt.subplots(1, 2, figsize=(12, 5))
83     ax[0].plot(bce_hist, label="BCE")
84     ax[0].plot(acc_hist, label="Accuracy")
85     ax[0].set_xlabel("Epochs")
86     ax[0].set_ylim(0, 1)
87     for n, grads in enumerate(grad_hist):
88         ax[1].plot(grads, label="layer"+str(n))
89     ax[1].set_xlabel("Epochs")
90     fig.suptitle(str(activation))
91     ax[0].legend()
92     ax[1].legend()
93     plt.show()

```

## Further Readings

This section provides more resources on the topic if you are looking to go deeper.

- [nn.Sigmoid](#) from PyTorch documentation
- [nn.Tanh](#) from PyTorch documentation
- [nn.ReLU](#) from PyTorch documentation
- [nn.ReLU6](#) from PyTorch documentation
- [nn.LeakyReLU](#) from PyTorch documentation
- [Vanishing gradient problem](#), Wikipedia

## Summary

In this chapter, you discovered how to select activation functions for your PyTorch model. You learned:

- What are the common activation functions and how they look like
- How to use activation functions in your PyTorch model
- What is vanishing gradient problem
- The impact of activation function to the performance of your model

## Get Started on Deep Learning with PyTorch!

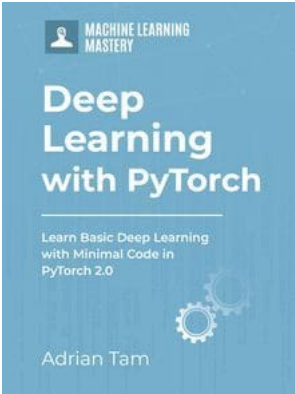
### Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:  
[Deep Learning with PyTorch](#)

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

**Kick-start your deep learning journey with hands-on exercises**



SEE WHAT'S INSIDE

Share Share Post

More On This Topic



Using Activation Functions in Neural Networks



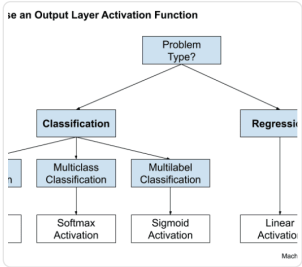
Linear Layers and Activation Functions in Transformer Models



Activation Functions in PyTorch



A Gentle Introduction to Activation Regularization...



How to Choose an Activation Function for Deep Learning



Softmax Activation Function with Python



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.  
View all posts by Adrian Tam →

< Save and Load Your PyTorch Models

Loss Functions in PyTorch Models >

2 Responses to *Using Activation Functions in Deep Learning Models*

**Evren** October 19, 2023 at 9:51 am #

REPLY ↩

Hi,

This is very useful tutorial for me thank you very much!

But these 2 statements are not clear for me can you explain further please

- 1) in backpropagation, passing the loss metric from the output layer all the way to the input layer
- 2) maintaining stable gradient calculation under specific condition, e.g., limiting floating point precision

**James Carmichael** October 20, 2023 at 10:29 am #

REPLY ↩

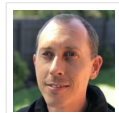
Hi Evren... You are very welcome! The following resource may be of interest to you:

<https://machinelearningmastery.com/implement-backpropagation-algorithm-scratch-python/>

## Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT

**Welcome!**

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

## Never miss a tutorial:



## Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python

[LSTM for Time Series Prediction in PyTorch](#)



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

## Loving the Tutorials?

The Deep Learning with PyTorch EBook  
is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate [website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved